

ΓΚ802 - Express.js

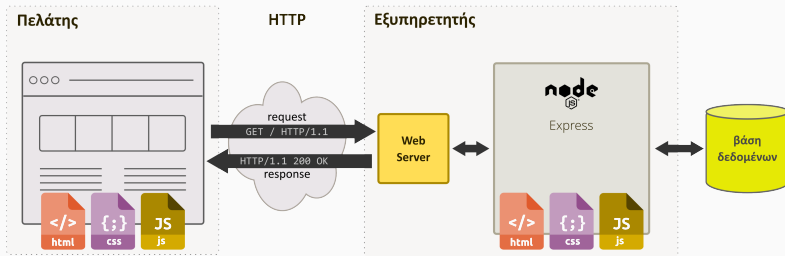
Ν. Αβούρης - Χ. Φείδας - Χ. Σιντόρης

2024

ECE/UPatras

Express.js

Το μοντέλο πελάτη-εξυπηρετητή



Σχήμα 1: Το μοντέλο πελάτη-εξυπηρετητή

Το [Express.js](#) είναι μια βιβλιοθήκη της Node.js.

Μάς βοηθά να αναπτύξουμε εφαρμογές Node.js στην πλευρά του εξυπηρετητή.

Διευκολύνει τη διεκπεραίωση βασικών εργασιών:

- HTTP Request parsing, Cookie parsing
- Session management
- Request routing
- Send HTTP Response
- Error handling
- ...

Express.js

Fast, unopinionated, minimalist web framework for [Node.js](#)

Εγκατάσταση του βασικού Express.js

Το Express.js είναι ένα απλό module της Node.js και εγκαθίσταται με τον γνωστό τρόπο.

Αρχικοποιούμε, αν δεν το έχουμε κάνει, το πρότζεκτ μας:

```
npm init -y
```

Εγκαθιστούμε το πακέτο express και τις εξαρτήσεις του:

```
npm install express
```

Μια βασική εφαρμογή Express.js

Αποθηκεύουμε στο αρχείο `index.mjs` το παρακάτω περιεχόμενο:

```
import express from 'express'
// Το "application object", το "αντικείμενο του Express.js"
const app = express()
const port = 3000

app.get('/', (req, res) => res.send('Γειά σου express!'))
app.listen(port, () => console.log('Έτοιμο!
↳ (http://localhost:${port}'))
```

- Στη γρ. εντολών τρέχουμε `node index.mjs`.
- Ανοίγουμε το φυλλομετρητή στη σελίδα <http://localhost:3000>.

Middleware

Middleware pattern

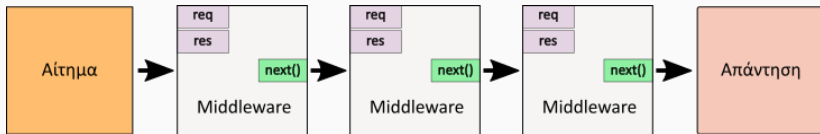
Ο εξυπηρετής του προηγούμενου παραδείγματος είναι πολύ απλός:

```
import express from 'express'
...
app.get('/', (req, res) => res.send('Γειά σου express!'));
...
```

Συνήθως οι απαιτήσεις μας είναι πολύ περισσότερες.

Το Express.js χρησιμοποιεί εκτεταμένα το **middleware pattern**, δηλ. τη σύνθεση του προγράμματος σαν μια **αλληλουχία υποτμημάτων** (το λεγόμενο “*pipeline*”).

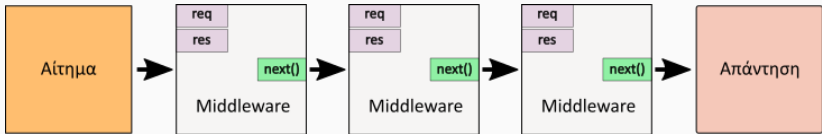
Η αλυσίδα επεξεργασίας



Κάθε υποτμήμα που δέχεται ένα αίτημα HTTP μπορεί να επεξεργάζεται και να μεταβάλλει το **request** και το **response** object και γενικά να εκτελεί κάποιο κώδικα, π.χ. για

- καταγραφή (logging),
 - αυθεντικοποίηση (authentication),
 - επεξεργασία του αιτήματος, ... κλπ
- και στη συνέχεια ...

Η αλυσίδα επεξεργασίας (pipeline) 2



... στη συνέχεια μπορεί να κάνει ένα από τα εξής:

- να δώσει τον έλεγχο στο επόμενο υποτμήμα της αλυσίδας επεξεργασίας με **next()**, ή
- να στείλει μια απόκριση στο αίτημα του πελάτη. Αν έπειτα κληθεί η **next()**, η αλυσίδα συνεχίζει να εκτελείται, αν όχι τερματίζει, ή
- να προκληθεί σφάλμα και να ξεκινήσει η αλυσίδα χειρισμού σφαλμάτων.

Τι είναι ένα middleware του Express.js;

Ένα υποτμήμα του Express.js (middleware) είναι απλά **μια συνάρτηση**, με τρεις παραμέτρους: (**req**, **res**, **next**).

Οι παράμετροι ονομάζονται από σύμβαση **req**, **res**, και **next**.

- **req**: Το αντικείμενο του **αιτήματος** HTTP που δεχτήκαμε από τον πελάτη.
- **res**: Το αντικείμενο της **απόκρισης** που τελικά θα αποσταλεί στον πελάτη.
- **next**: Το επόμενο middleware της αλυσίδας.

Για να τοποθετήσουμε ένα middleware στην αλυσίδα επεξεργασίας, χρησιμοποιούμε το **application object**, το **app**:

```
app.use(<όνομα συνάρτησης middleware>)
```

Παράδειγμα χρήσης ενός middleware

Το `myMiddleware()` είναι απλά μια συνάρτηση. Δηλώνουμε πως είναι μέρος της αλυσίδας επεξεργασίας με `app.use()` (γίνεται και αλλιώς όπως θα δούμε).

```
import express from 'express'
const app = express()

const myMiddleware = (req, res, next) => {
  console.log('my middleware')
  next()
}

app.use(myMiddleware)
```

Το `express.js` θα τερματίσει την αλυσίδα επεξεργασίας με την κλήση της `next()`, καθώς δεν υπάρχει άλλο middleware.

Η σειρά έχει σημασία

Η σειρά με την οποία δηλώνουμε τα middleware έχει σημασία:

```
const myMiddleware1 = (req, res, next) => {  
  console.log('MW-1')  
  next()  
}  
  
const myMiddleware2 = (req, res, next) => {  
  console.log('MW-2')  
  next()  
}  
  
app.use(myMiddleware2)  
app.use(myMiddleware1)  
  
// ή app.use(myMiddleware2, myMiddleware1, ... κλπ)
```

Middleware με `app.use()`

Η χρήση της `app.use()` στο προηγούμενο παράδειγμα, σημαίνει και ότι αυτά τα υποτμήματα θα εκτελεστούν σε **κάθε** αίτημα HTTP.

Στο τερματικό θα εμφανιστούν τα μηνύματα με αυτή τη σειρά, ανεξάρτητα από τί ζήτησε ο χρήστης στο αίτημά του:

```
...  
app.use(myMiddleware2)  
app.use(myMiddleware1)  
// θα εκτυπώσει:  
MW-2  
MW-1
```

Middleware app.use() 2

Την ίδια σειρά επεξεργασίας μπορούμε εναλλακτικά να ορίσουμε περνώντας μια σειρά από υποτιμήματα σαν παραμέτρους στην **app.use()**:

```
app.use(myMiddleware2, myMiddleware1, myMiddleware3)
```

ή χρησιμοποιώντας, για ευκολία, έναν πίνακα:

```
const pipeline = [myMiddleware2, myMiddleware1, myMiddleware3]  
app.use(pipeline)
```

Middleware - τερματισμός της αλυσίδας επεξεργασίας

Η αλυσίδα επεξεργασίας τερματίζει όταν συμβεί κάτι από τα εξής:

1. αν ένα υποτμήμα της αλυσίδας δεν καλέσει τη **next()**, η αλυσίδα τερματίζει εκεί και καμία απάντηση δεν στέλνεται στον πελάτη - δεν το θέλουμε.



Φροντίζουμε κάθε αίτημα να λαμβάνει απόκριση, δηλ. να μην τερματίζει η αλυσίδα χωρίς να σταλεί απόκριση.

Middleware - τερματισμός της αλυσίδας επεξεργασίας

Η αλυσίδα επεξεργασίας τερματίζει όταν συμβεί κάτι από τα εξής:

1. αν ένα υποτμήμα της αλυσίδας δεν καλέσει τη `next()`, η αλυσίδα τερματίζει εκεί και καμία απάντηση δεν στέλνεται στον πελάτη - δεν το θέλουμε.
2. αν ένα υποτμήμα τη αλυσίδας στείλει απάντηση στον πελάτη και δεν καλέσει τη `next()`. Αυτό μπορεί να γίνει με κλήση μεθόδων όπως `res.send()`, `res.json()`, `res.render()`.



Φροντίζουμε κάθε αίτημα να λαμβάνει απόκριση, δηλ. να μην τερματίζει η αλυσίδα χωρίς να σταλεί απόκριση.

Middleware - τερματισμός της αλυσίδας επεξεργασίας

Η αλυσίδα επεξεργασίας τερματίζει όταν συμβεί κάτι από τα εξής:

1. αν ένα υποτμήμα της αλυσίδας δεν καλέσει τη `next()`, η αλυσίδα τερματίζει εκεί και καμία απάντηση δεν στέλνεται στον πελάτη - δεν το θέλουμε.
2. αν ένα υποτμήμα τη αλυσίδας στείλει απάντηση στον πελάτη και δεν καλέσει τη `next()`. Αυτό μπορεί να γίνει με κλήση μεθόδων όπως `res.send()`, `res.json()`, `res.render()`.
3. αν συμβεί ένα σφάλμα και δεν το χειριστούμε. Αυτό θα δώσει τον έλεγχο στην αλυσίδα χειρισμού σφαλμάτων, αν υπάρχει.



Φροντίζουμε κάθε αίτημα να λαμβάνει απόκριση, δηλ. να μην τερματίζει η αλυσίδα χωρίς να σταλεί απόκριση.

Δρομολόγηση

Δρομολόγηση (routing)

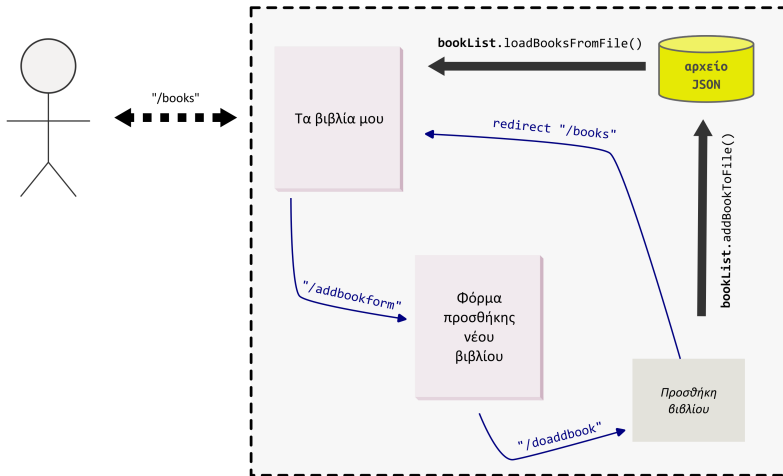
Σπάνια όμως θέλουμε η αλυσίδα επεξεργασίας (*pipeline*) να αποτελείται πάντα από τα ίδια middleware, ανεξάρτητα από το αίτημα του χρήστη.

Συνήθως θέλουμε να εκτελεστούν άλλα υποτμήματα, ανάλογα με το αίτημα του πελάτη.

Μπορεί ένα αίτημα να αφορά π.χ.

- αυθεντικοποίηση, ή
- προβολή πληροφορίας που αντλήσαμε από τη ΒΔ, ή
- προβολή μιας φόρμας για την υποβολή στοιχείων, ή
- υποδοχή των των στοιχείων που υποβλήθηκαν με τη φόρμα, ή
- κλπ ...

Δρομολόγηση (routing) 2



Σχήμα 2: Η απλή εφαρμογή διαχείρισης βιβλίων.

Δρομολόγηση (routing) 3

Η **δρομολόγηση** προδιαγράφει ποιο τμήμα της εφαρμογής θα αποκριθεί στο αίτημα.

Καθορίζει πώς θα αποκριθεί η εφαρμογή Express.js σε αιτήματα πελατών.

Μια **διαδρομή** (*route*) καθορίζεται από:

- Τη **μέθοδο HTTP** που χρησιμοποιήθηκε στο αίτημα.
- Το μονοπάτι (**path**) του αιτήματος που έλαβε η εφαρμογή μας.

α) Δρομολόγηση (routing) με το application object

Ο πιο απλός τρόπος δρομολόγησης είναι απευθείας στο αντικείμενο του Express.js με την **app.use()**:

```
app.use("/path1", myMiddleware2)
app.use("/path2", myMiddleware1)
app.use(myMiddleware3)
```

Εδώ έχουμε 3 διαφορετικές αλυσίδες επεξεργασίας:

μονοπάτι αιτήματος

αλυσίδα επεξεργασίας

αίτημα στο **/path1**

→ **myMiddleware2** → **myMiddleware3** →

αίτημα στο **/path2**

→ **myMiddleware1** → **myMiddleware3** →

οτιδήποτε άλλο μονοπάτι

→ **myMiddleware3** →

α) Δρομολόγηση (routing) με το application object #2

Πάλι στο αντικείμενο του Express.js, αυτή τη φορά με βάση τη μέθοδο HTTP:

```
app.METHOD(path, callback)
```

όπου

- **METHOD** είναι μια από τις μεθόδους HTTP (π.χ. **get**, **post** κλπ),
- **path** είναι το μονοπάτι για το οποίο θα ισχύει η δρομολόγηση αυτή,
- **callback** είναι η συνάρτηση που θα εκτελεστεί για το συνδυασμό **METHOD+path**.

Παράδειγμα

Η αλυσίδα επεξεργασίας θα ενεργοποιηθεί με **get** στο **/**.

Θα αποτελείται από δύο τμήματα (τουλάχιστον):

```
app.get('/', (req, res, next) => {  
    console.log(req.originalUrl)  
    next()  
},  
(req, res, next) => {  
    res.send('Hello World!'),  
})
```

“*Τουλάχιστον*”, γιατί μπορεί να έχουμε δηλώσει middleware με **app.use(middleware)**.

β) Δρομολόγηση (routing) - με τη μέθοδο `app.route()`

Επιτρέπει την κατασκευή μιας αλυσίδας χειρισμού του αιτήματος σε ένα συγκεκριμένο URI (παράδειγμα από την [τεκμηρίωση](#)):

```
app.route('/book')  
  .get(function (req, res) {  
    res.send('Get a random book')  
  })  
  .post(function (req, res) {  
    res.send('Add a book')  
  })  
  .put(function (req, res) {  
    res.send('Update the book')  
  })
```

γ) Δρομολόγηση με την κλάση Router

Δρομολόγηση με το αντικείμενο **Router** γίνεται όπως και με το το αντικείμενο του Express.js.

Η διαφορά του αντικ. **Router** με το αντικείμενο του Express.js είναι ότι υποστηρίζει μόνο το μηχανισμό middleware και τα routes. Αυτή η μέθοδος βοηθά στην οργάνωση όταν η εφαρμογή γίνεται πιο σύνθετη.

```
const router = express.Router()

router.get('/random', function (req, res) {
  res.send('Get a random book')
})

app.use('/book', router)
```

Η διαδρομή είναι η **/book/random**.

Δρομολόγηση (routing)

Θεωρούνται ίδιες οι διαδρομές (routes)

- με ή χωρίς τελικό '/', π.χ. αίτημα στο /books είναι το ίδιο με /books/,
- με μικρά ή κεφαλαία, π.χ. /books το ίδιο με /Books.

Η συμπεριφορά αυτή αλλάζει με τα αντίστοιχα options, strict και caseSensitive:

```
const options = {  
  caseSensitive: true,  
  strict: true  
}  
  
const router = express.Router(options)
```

Δρομολόγηση (routing) - HTTP request methods

Όλες οι μέθοδοι HTTP είναι διαθέσιμες στο αντικείμενο Application ([Application API](#)):

```
app.get('/', (req, res) => { /* */ })
app.head('/', (req, res) => { /* */ })
app.post('/', (req, res) => { /* */ })
app.put('/', (req, res) => { /* */ })
app.delete('/', (req, res) => { /* */ })
app.patch('/', (req, res) => { /* */ })
```

- Στο **request object** req έχουμε το **αίτημα** του πελάτη.
- Στο **response object** res γράφουμε όλη την **απάντηση** που θα στείλουμε πίσω στον πελάτη.

Η μέθοδος `app.all()` ορίζει έναν χειριστή για όλες τις μεθόδους (χρήσιμο π.χ. για να εξασφαλίσουμε πως ο χρήστης έχει αυθεντικοποιηθεί).

To Request object

Το αντικείμενο Request

Στο *request object* **req** έχουμε πληροφορίες για το **αίτημα** του πελάτη που έφτασε στο υποτμήμα επεξεργασίας (middleware).

```
app.get('/', (req, res, next) => {  
  ...  
  ...  
})
```

Για παράδειγμα, **req.query** περιέχει το “*query string*”.

Το αντικείμενο Request #2

```
import express from 'express'
const app = express()
app.get('/recipe/', function (req, res) {
  for (const key in req.query)
    console.log(key, ":", req.query[key])
  res.send("got a recipe.")
})
app.listen(3000, () => console.log('Ready'))
```

Το <http://localhost:3000/recipe?diff=easy&rating=5> θα τυπώσει:

diff : easy

rating : 5

Το αντικείμενο Request #3

Το API του αντικειμένου Request μας δίνει πρόσβαση στα δεδομένα του αιτήματος ([Request API](#)):

- **request.query:** οι παράμετροι που πέρασαν στο query string,
- **request.method:** η μέθοδος που χρησιμοποιήθηκε,
- **request.body:** το σώμα του αιτήματος,

Ονοματισμένες παράμετροι - Named parameters

Μπορούμε να περάσουμε παραμέτρους στο URI με το μηχανισμό των **ονοματισμένων παραμέτρων** - *named parameters*.

```
app.get('/recipe/difficulty/:diff/rating/:rating', (req, res) => {  
  console.log(req.params)  
  res.send("got a recipe.")  
})
```

Η κλήση στο URL <http://localhost:3000/recipe/diff/easy/rating/5> θα έχει σαν αποτέλεσμα:

```
{ diff: 'easy', rating: '5' }
```

Πρόσβαση στα δεδομένα POST

Για να έχουμε πρόσβαση στα δεδομένα που έχουν σταλεί με τη μέθοδο **POST**, πχ. από μια φόρμα, χρειάζεται να ενεργοποιήσουμε το ενσωματωμένο middleware **urlencoded** [↗](#):

```
app.use(express.urlencoded({ extended: true }))

app.post('/', (req, res) => {
  console.log("request body:", req.body)
  res.send("post ok")
})
```



Το `{ extended: true }` προσδιορίζει με [ποια βιβλιοθήκη θα γίνει η αποκωδικοποίηση του query string](#) [↗](#). Συνοπτικά, με την τιμή `true` μπορούμε να περάσουμε σύνθετα αντικείμενα στο αίτημά μας.

To Response object

Στο **response object** `res` γράφουμε όλη την **απάντηση** που θα στείλουμε πίσω στον πελάτη.

- **`response.send()`**: Υπολογίζει το μέγεθος της απάντησης (Content-Length), τη στέλνει, και κλείνει τη σύνδεση.
- **`response.end()`**: Στέλνει κενή απάντηση (χωρίς body), κλείνει τη σύνδεση.
- **`response.status()`**: Ορίζει την κεφαλίδα κατάστασης της απάντησης: `res.status(404).end()`;

Απάντηση σε μορφή JSON:

- `Response.json({name: 'Maria'})`

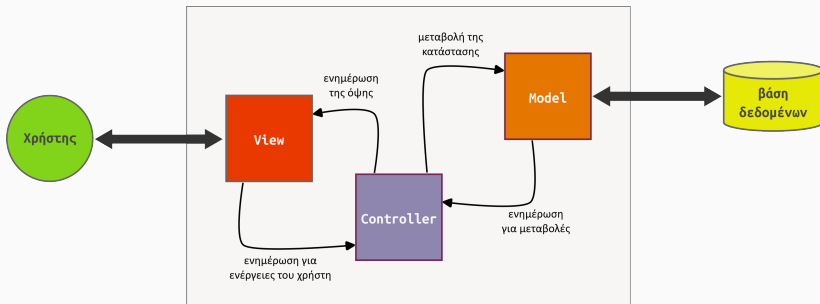
Η αρχιτεκτονική MVC (model-view-controller)

Η αρχιτεκτονική MVC (model-view-controller)

Η αρχιτεκτονική οργάνωσης του κώδικα **MVC** (model-view-controller) προβλέπει τρία διακριτά τμήματα του κώδικα,

- το τμήμα *Model* (Μοντέλο), το οποίο χειρίζεται τη λογική των δεδομένων,
- το τμήμα *View* (Προβολή), το οποίο εμφανίζει τις πληροφορίες από το μοντέλο στο χρήστη, και
- το τμήμα *Controler* (Ελεγκτής), που ελέγχει τη ροή δεδομένων σε ένα αντικείμενο μοντέλου και ενημερώνει την προβολή κάθε φορά που αλλάζουν τα δεδομένα.

Η αρχιτεκτονική MVC 2



Σχήμα 3: Η αρχιτεκτονική MVC.

Η αρχιτεκτονική MVC 3

Μια εφαρμογή που ακολουθεί την αρχιτεκτονική MVC θα μπορούσε να είναι διαρθρωμένη ως εξής:

Το `app.js` είναι το σημείο εισόδου της εφαρμογής. Στη συνέχεια, το πρόγραμμα μοιράζεται σε διάφορα τμήματα:

- `/model` παρέχει την πρόσβαση στα δεδομένα.
- `/controller` περιέχει συναρτήσεις για να έχουμε πρόσβαση στα δεδομένα που μας δίνει το `model`.
- Οι φάκελοι `views` και `routes`:
 - Ο `/views` περιέχει `template` (γραμμένα σε `handlebars`).
 - Ο `/routes` περιέχει τις διαδρομές που αναγνωρίζει η εφαρμογή.

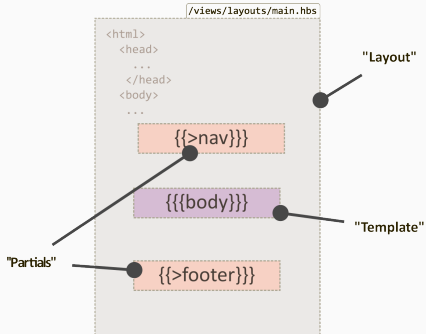
Templates

Με τις μηχανές για templates έχουμε απλές γλώσσες για τη δυναμική δημιουργία HTML.

Η μηχανή template αναλαμβάνει να συνθέσει και να αποστείλει στον πελάτη μια σελίδα HTML που να περιέχει το αποτέλεσμα της επεξεργασίας του αιτήματός του.

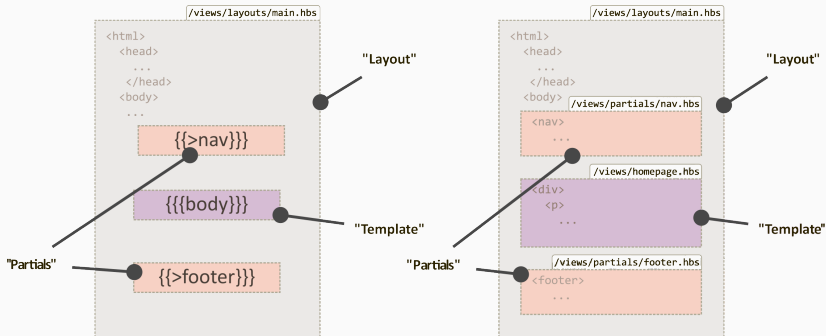
Η επίσημη μηχανή για templates στο Express.js είναι η [Pug](#). Η πιο δημοφιλής είναι η [Handlebars.js](#).

Handlebars - Δομή αρχείων



Σχήμα 4: Handlebars - Δομή αρχείων

Handlebars - Δομή αρχείων



Σχήμα 5: Handlebars - Δομή αρχείων

Handlebars - Δομή αρχείων 2

Η βασική δομή του πρότζεκτ μας θα είναι:

```
|-- index.js
|-- views          // η προκαθορισμένη θέση των template
    |-- home.hbs
    |-- layouts
        |-- main.hbs // το κύριο layout, μέσα στο οποίο
        |             // προβάλλονται τα template
|-- partials       // επιμέρους αποσπάσματα template,
    |             // για να φορτώνονται από άλλα template
    |-- header.hbs
    |-- footer.hbs
```

Handlebars - Εγκατάσταση και χρήση

Με το πακέτο [express-handlebars](#).

```
npm install express-handlebars
```

Χτίζουμε την εφαρμογή μας και ρυθμίζουμε το handlebars.

```
// Αρχείο index.mjs
import express from 'express';
import { engine } from 'express-handlebars';

const app = express()

// Δηλώνουμε πως θα χρησιμοποιήσουμε το handlebars
// και αλλάζουμε την προκαθορισμένη επέκταση σε .hbs.
app.engine('.hbs', engine({ extname: '.hbs' }));
app.set('view engine', '.hbs');
```

Handlebars - Layout 1/2

Το template μας μπορεί να χωριστεί σε πολλά τμήματα.

Το default layout `/views/layouts/main.hbs`, περιέχει το βασικό layout μέσα στο οποίο θα προβληθεί το template:

```
<!-- αρχείο /views/layouts/main.hbs -->
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Hello Express Handlebars</title>
</head>
<body>{{{body}}}</body>
</html>
```

Το εκάστοτε template θα προβληθεί στη θέση `{{{body}}}`.

Handlebars - Template rendering

Το template `/views/home.hbs` μπορεί να είναι πολύ απλό:

```
<main><p> My home page </p></main>
```

Συνήθως μεταφέρουμε στο template δεδομένα για να προβληθούν.
Το κάνουμε περνώντας τα σε ένα αντικείμενο:

```
app.get('/', (req, res) => {  
  // θα κάνει render το template `views/home.hbs`, περνώντας τη  
  ↪ μεταβλητή `greeting`  
  res.render('home', { greeting: 'Hello!' });  
});
```

Το αποτέλεσμα θα προστεθεί στο default layout
(`/views/layouts/main.hbs`) και θα επιστραφεί στον πελάτη:

```
<main><p> My home page: {{greeting}} </p></main>
```

Handlebars - Layout 2/2

Στον φάκελο **/views/layouts/**, μπορούμε να έχουμε και άλλα layouts:

```
|-- views
  |-- layouts
    |-- main.hbs
    |-- other-layout.hbs
```

Π.χ. να γίνει render το template **/views/home.hbs** στο layout **/views/layouts/other-layout.hbs**:

```
app.get('/page-with-different-layout', (req, res) => {
  res.render('home', { layout: "other-layout" })
})
```

Handlebars - Partials

Τα [partials](#) επιτρέπουν να φορτώσουμε επιμέρους αποσπάσματα templates μέσα σε άλλα templates.

π.χ. το partial `/views/partials/footer.hbs` περιέχει

```
<footer><p> My footer </p></footer>
```

Το φορτώνουμε στο layout μας (`/views/layouts/main.hbs`) με:

```
...  
{{> footer}}  
</body>
```

Πώς όμως θα διαθέσουμε στους πελάτες του εξυπηρετητή αρχεία που δεν χρειάζονται επεξεργασία;

Τέτοια αρχεία είναι για παράδειγμα

- εικόνες,
- αρχεία CSS,
- αρχεία με JavaScript που εκτελείται στο φυλλομετρητή.

Αυτά ονομάζονται *στατικά αρχεία* και για να τα διαθέσουμε χρησιμοποιούμε ένα από τα ενσωματωμένα middleware του Express.js, το **static**.

Το ενσωματωμένο middleware static

Ορίζουμε τον φάκελο που περιέχει [στατικά](#) αρχεία:

```
app.use(express.static('public')) // φάκελος /public
```

Τα αρχεία στο φάκελο αυτό είναι προσβάσιμα απευθείας, π.χ.

```
|-- public
  |-- images
    |-- mypic.png
  |-- css
    |-- mystyle.css
  ...
```

Τα αρχεία είναι προσβάσιμα με, π.χ.:

- <http://localhost:3000/images/mypic.png>
- <http://localhost:3000/images/mystyle.css>

Handlebars - Expressions

Οι εκφράσεις περικλείονται από διπλές αγκύλες `{{greeting}}`, με dot-notation, π.χ. για το αντικείμενο **address**:

Π.χ., το παρακάτω αντικείμενο

```
{  address: {  street: "Υπατίας 1", city: "Patra" } }
```

μπορεί να χρησιμοποιηθεί στο template ως

```
<p>
  {{address.street}} {{address.city}}
</p>
```

Handlebars - Built-in helpers - with

Οι εκφράσεις περικλείονται από διπλές αγκύλες `{{greeting}}`, με dot-notation, π.χ. για το αντικείμενο **address**:

Π.χ., το παρακάτω αντικείμενο

```
{  address: {  street: "Υπατίας 1", city: "Patra" } }
```

ή με **with**, να περιορίσουμε το evaluation context του μπλοκ

```
{{#with address}}  
<p> {{street}} {{city}}</p>  
{{/with}}
```

Handlebars - Built-in helpers - each

Οι εκφράσεις περικλείονται από διπλές αγκύλες `{{greeting}}`, με dot-notation, π.χ. για το αντικείμενο **address**:

Π.χ., το παρακάτω αντικείμενο

```
{  address: {  street: "Ypatias 1", city: "Patra" } }
```

ή με **each** να διατρέξουμε τις ιδιότητες του αντικειμένου

```
{{#each address}}  
<p> {{@key}}: {{this}}</p>  
{{/with}}
```

Handlebars - Built-in helpers - if/unless

Η handlebars μας δίνει και άλλα βοηθήματα ([helpers](#)):

- [#if](#):

```
{{#if greeting}}  
  {{greeting}}  
{{else}}  
  Goodbye  
{{/if}}
```

- [#unless](#): το αντίθετο του #if